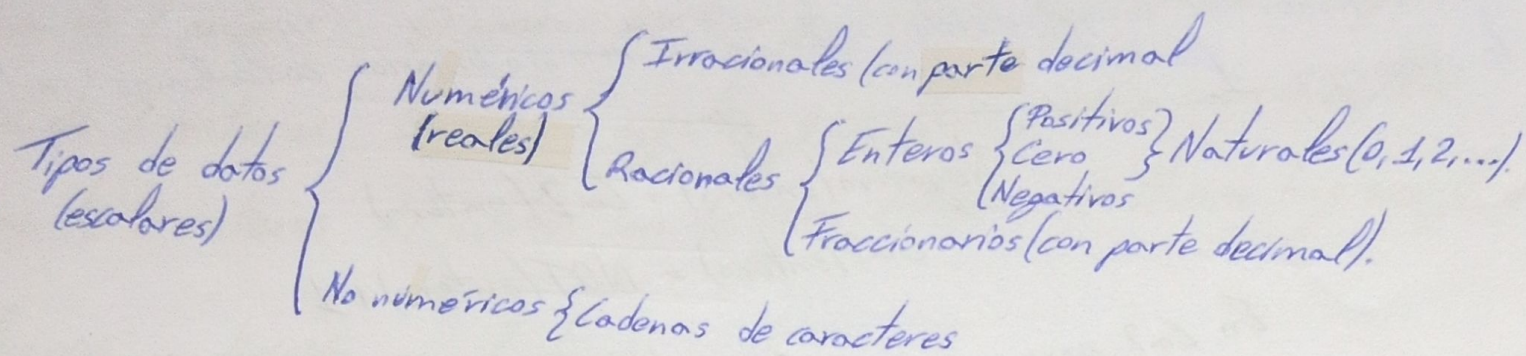


Aritmética binaria

Tipos de datos



Representación de los números naturales y enteros

Los números positivos son iguales en todas las formas representativas.

Signo y magnitud.

Los números enteros tienen dos componentes: el signo y la magnitud, se requiere de un bit para indicar el signo.

Si se dispone de un número binario de n dígitos, uno está dedicado al signo, el resto $(n-1)$ a la magnitud, por lo que se podrán expresar 2^{n-1} valores positivos y 2^{n-1} valores negativos.

bit de signo { 0 ← positivo
 1 ← negativo
0001101
 magnitud.

- Rango simétrico de positivos y negativos.
- Operaciones aritméticas complejas
- Doble representación del cero { 0000
 1000

Complemento a 1. ("complemento a la base restringida" o "complemento a la base menos 1").

Se requieren dos componentes:

- Bit de signo (el de mayor peso).
- Bits restantes $(n-1)$ representan la magnitud del valor.

Para convertir un número binario positivo a negativo se le aplica la operación lógica NOT.

$$\begin{aligned}\text{Negativo (entero)} &= \text{Ca1 (entero)} \\ \text{Ca1 (entero)} &= \text{NOT (entero)}.\end{aligned}$$

En Ca1 para convertir un número n { -positivo a negativo $\rightarrow \text{NOT}(n)$
 -negativo a positivo $\rightarrow \text{NOT}(n)$
 (conocer su magnitud).

- Rango simétrico de positivos y negativos.
- Facilidad en las operaciones aritméticas, se suma el acarreo final.
- Doble representación del cero { 0000
 1111

Complemento a 2. ("complemento a la base").

El bit de mayor peso indica el signo (0, positivo; 1, negativo), el resto la magnitud.

Para calcular el complemento a 2 de un número binario calculamos su NOT y le sumamos 1.

$$\text{Negativo}(\text{entero}) = \text{Ca2}(\text{entero})$$

$$\text{Ca2}(\text{entero}) = \text{NOT}(\text{entero}) + 1$$

En Ca2 para convertir un número n

$$\begin{cases} \text{- positivo a negativo} \rightarrow \text{NOT}(n) + 1. \\ \text{- negativo a positivo} \rightarrow \text{NOT}(n) + 1. \end{cases}$$

Rango de 2^{n-1} negativo a $2^{n-1} - 1$ positivo, la combinación que se asignaba doble al cero en las anteriores representaciones se usa para el número más negativo.

- Rango asimétrico de positivos y negativos
- Facilidad en las operaciones aritméticas, se ignora el acarreo final.
- No hay doble representación del cero.

Otras operaciones con números enteros.

La resta.

$$A - B = A + (-B)$$

Aquí la utilidad de los números negativos expresados en Ca1 y Ca2, para convertir un número en su opuesto, se convierte en su complemento (Ca1 o Ca2).

050 Para dos valores naturales o enteros positivos A y B , tal que $A \geq B$ si el valor de A se le resta B se produce acarreo! Este acarreo se desprecia para formar el resultado de la suma, sirve para indicar que el operando A es mayor o igual que B .

Acarreo y overflow.

Si se superan los bits dispuestos para el resultado pero el último acarreo no al añadirse al resultado es correcto, no se toma como overflow.

Es overflow cuando se sobre pasa la representación y nos dan resultados erróneos de números positivos sumados dar como resultado negativo, o la suma de negativos dar como resultado positivo.

Overflow es una inversión coherente del signo en el resultado de una operación aritmética con números con signo.

Suma de 2 naturales Si el resultado $> 2^{n-1} \rightarrow$ Acaqueo
 Suma de 2 enteros Si el resultado $> 2^{n-1}$ o $< -2^{n-1} \rightarrow$ Overflow.
 Suma de 2 enteros Si ambos positivos y resultado negativo $\rightarrow$ Overflow
 Suma de 2 enteros Si ambos negativos y resultado positivo $\rightarrow$ Overflow
 Suma de 2 enteros Si ambos son de signo distinto $\rightarrow$ Nunca hay overflow

Multiplicación y división por la base.

Para multiplicar un número binario por 2, 4, 8, 16... $\rightarrow$ Basta con desplazar a la izquierda y añadir ceros a la derecha.
 Si es un entero y cambia de signo $\rightarrow$ Overflow

Para dividir un número binario por 2, 4, 8, 16... $\rightarrow$ Basta con desplazar a la derecha y añadir ceros a la izquierda.
 No hay parte fraccionaria, se trunca el resultado.

División de números negativos

Se vuelven positivos; las divisiones de enteros mediante desplazamientos se deben hacer "con extensión de signo"

$$10000 \div 2 = \boxed{11000}$$

OJO En Ca2, si el valor es negativo, se redondea al valor más negativo. El resultado no se trunca, sino que se redondea ($-5 \div 2 = -3$ por desplazamiento).

Representación de números reales

Coma flotante (IEEE 754).

Los números reales constan de $\begin{cases} \text{- Parte entera} \\ \text{- Parte fraccionaria} \end{cases}$

Uso de la notación exponencial

$$\text{mantisa} \times \text{base}^{\text{exp(entero)}}$$

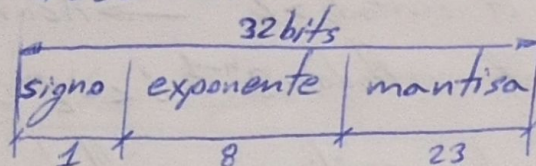
Normalización:

1. Eliminar ceros no significativos por la izquierda.
2. Mover la coma a la derecha del primer dígito significativo (solo un 1 antes de la coma).

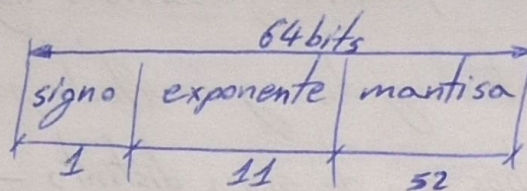
$$\text{Cantidad Real} = \pm \text{Mantisa}_{\text{base}} \times \text{base}^{\text{exponente}}$$

Seguendo el estándar de IEEE 754.

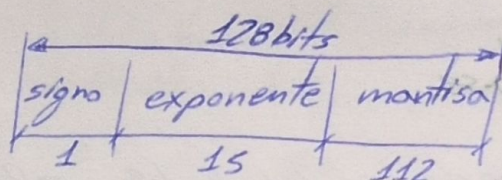
Simple precisión



Doble precisión



Cuadruple precisión



El IEEE 754 es formato estándar para la representación de los números reales en coma flotante. Se basan en una notación exponencial con base 2:

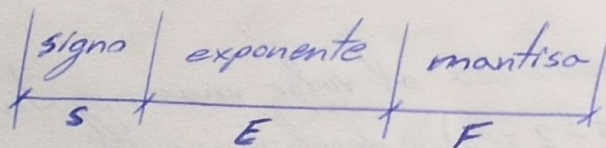
- Bit de signo: 0 positivo y 1 negativo.
- Mantisa: es un número natural (entero positivo) que constituye la parte fraccionaria una vez normalizado el número.
- Exponente: puede ser positivo o negativo, no obstante, aquí se representa como un número "en exceso a $2^{n-1}-1$ " (se obliga a ser positivo).

$$E = 2^{n-1} - 1 + \text{nº exponente en decimal normalizado.}$$

$\uparrow$ $n = \text{nº bits dedicados al exponente.}$
 $\uparrow$ exponente en exceso

El exponente en exceso no puede tomar ni todos 0 ni todos 1 (tabla de excepciones).

Interpretación



$$\text{Exponente real} = E - 2^{n-1} - 1$$

$$\text{Valor} = (-1)^S \cdot (1, F \cdot 2^{E-2^{n-1}-1})$$

Simple precisión $\Rightarrow 127$

Doble precisión $\Rightarrow 1023$

Cuadruple precisión $\Rightarrow 16383$

Rango de representación

Simple precisión (2^{-126})

$$-(2-2^{-23}) \cdot 2^{127} \dots -2^{-126}, 0, 2^{-126} \dots (2-2^{-23}) \cdot 2^{127}$$

Doble precisión (2^{-1022})

$$-(2-2^{-52}) \cdot 2^{1023} \dots -2^{-1022}, 0, 2^{-1022} \dots (2-2^{-52}) \cdot 2^{1023}$$

Los valores 00000000 y 11111111 del exponente están reservados, quedan los valores 1 a 126 para exponentes negativos y del 128 al 254 para positivos, se puede indicar un exponente con el rango -126 a +127. La mantisa del mayor número positivo posible será $(2-2^{-23}) \cdot 2^{127}$ y el más negativo $-(2-2^{-23}) \cdot 2^{127}$.

Excepciones. Valores especiales

Exponente	Mantisa	Valor especial
Todos ceros $00...00$	$00...0$	Valor decimal ± 0 (según bit de signo).
	$\neq 0$	Número pequeño (desnormalizado).
Todos unos $11...11$	$00...00$	$+\infty$ y $-\infty$ (según bit de signo).
	$\neq 0$	Not a Number (NaN) $\left\{ \begin{array}{l} \text{- Valor indeterminado } \frac{0}{0}, \frac{\infty}{\infty}, 0 \cdot \infty \\ \text{- Operación inválida } (\sqrt{-n}) \end{array} \right.$

En coma flotante no todos los valores del exponente son válidos para representar diferentes números reales.

Cero. Cuando el exponente y la mantisa tienen el valor 0 indica el valor "cero", con el bit de signo puede ser cero positivo o negativo.

Infinito. Se utiliza para indicar que el resultado de una operación es superior al mayor número representable.

NaN. (Not a Number), indican que el valor no representa un número real.
 $\neq$ NaN operaciones inválidas.

Si el resultado de una operación es menor que el representable por la precisión utilizada por ese formato, se produce **underflow**. En este caso, se devuelve un valor "desnormalizado". Un valor desnormalizado no tiene la misma precisión que los números normales del mismo formato. Estos números no tienen el "1" antes de la coma binaria y, para precisión simple, se representan como $(-1)^s \cdot 0, M \cdot 2^{-126}$.

Rango dinámico. Reparto de bits. Precisión.

A mayor nº bits en el exponente $\rightarrow$ Mayor rango dinámico. $\rightarrow$ Diferencia entre el menor y el mayor nº representable.

A mayor nº bits en la mantisa $\rightarrow$ Mayor precisión. $\rightarrow$ Diferencia entre dos valores representables seguidos.

El IEEE 754 busca el punto de equilibrio con $\frac{1}{4}n$ bits en el exponente y $\frac{3}{4}n$ bits en la mantisa.

- Como con los números enteros, también hay overflow y underflow.
- Debido a su precisión limitada, a menudo se requiere terminar haciendo redondeos.
- Algunos valores fraccionarios simples como $\frac{1}{3}$ no tienen representación decimal, pues requieren infinitos decimales. En la base binaria hay muchos valores simples que tiene representación decimal exacta (0,1; 0,2; 0,3; 0,4...) pero no la tienen en coma flotante.
- Debido a falta de precisión infinita, pueden resultar diferentes, por lo que al compararlos por igualdad pueden dar un resultado falso.

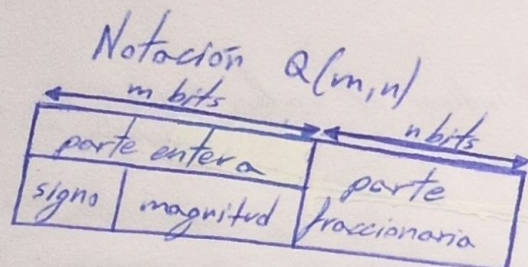
- Dependiendo del bit de signo el valor "cero" puede ser positivo o negativo. Aunque se comportan igual en comparaciones numéricas, pueden producir resultados distintos en algunas ocasiones.

Representación de números reales

Como fija.

Número real = entero $\times$ "factor de escala"

El usuario-programador establece el número de bits de la parte entera y la parte fraccionaria. A partir de entonces, los operandos que utilice con el tipo definido y los resultados que se produzcan tendrán ese formato.



Misma notación que los enteros
(SyM, Ca1, Ca2)
"Mas la coma".

Operaciones en coma fija

1. Se normaliza el número quitando la coma y guardando el factor de la escala, siguiendo el formato acordado.
2. Se realiza la operación, en las sumas y las restas, el factor de escala del resultado es igual al de los operandos normalizados.
3. Resultado con coma y formato adaptado sin redundancia. Comprobación del signo.

En restas, si el resultado entero de la suma/resta es negativo, antes de aplicarle el factor de escala para representar el resultado, debe hacerse el Ca2 y a continuación aplicar el factor de escala.

Rango y resolución.

Sin signo
 $[0 \rightarrow 2^m - 2^{-n}]$

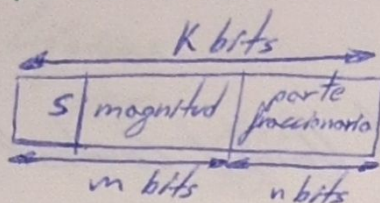
SyM
 $[-(2^{m-1} - 2^{-n}) \rightarrow 2^{m-1} - 2^{-n}]$

Ca1
 $[-(2^{m-1} - 2^{-n}) \rightarrow 2^{m-1} - 2^{-n}]$

Ca2
 $[-2^{m-1} \rightarrow 2^{m-1} - 2^{-n}]$

$\frac{1}{2^n} = \Delta$ (Resolución)
 distancia entre números consecutivos
 $n = \text{nº bits de la parte fraccionaria.}$

Reparto de bits.



Siendo K fijo

Si $m \uparrow \Rightarrow n \downarrow$
Aumenta rango
Disminuye resolución

Si $m \downarrow \Rightarrow n \uparrow$
Aumenta resolución
Disminuye rango

Sin signo

$$0 \rightarrow (2^k - 1)/2^n$$

Co2

$$-(2^{k-1})/2^n \rightarrow (2^{k-1})/2^n$$

Sy M

$$-(2^{k-1})/2^n \rightarrow (2^{k-1})/2^n$$

Co1

$$-(2^{k-1})/2^n \rightarrow (2^{k-1})/2^n$$

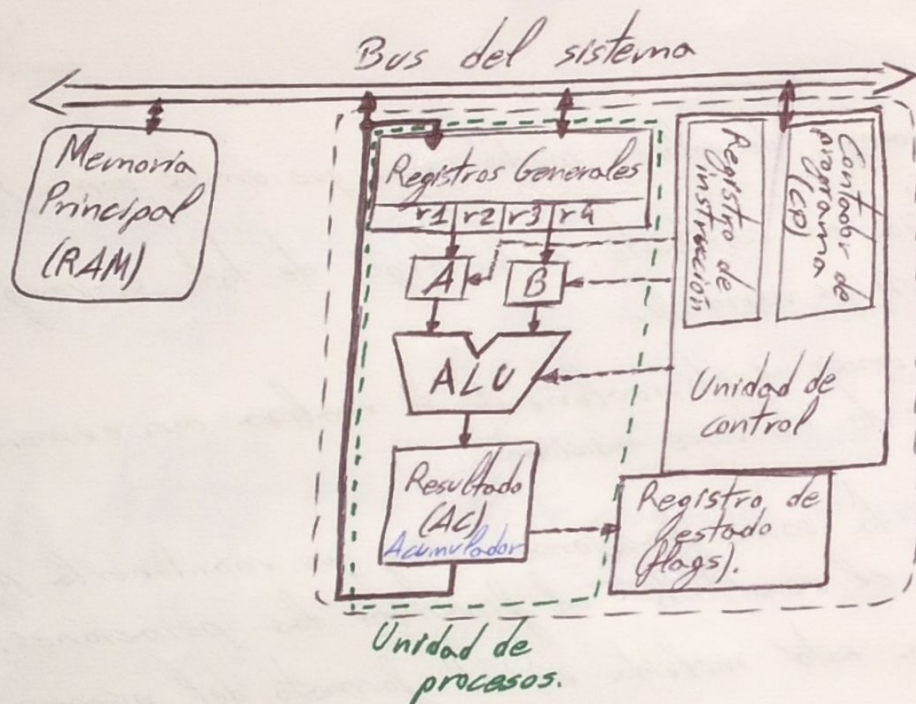
- La coma flotante ofrece un rango de valores mucho mayor que en la coma fija.
- La precisión en la coma fija es constante a lo largo de todo su rango de valores. En coma flotante, la precisión es variable.
- El cálculo en coma fija es más rápido, realmente se realiza con números reales. En coma flotante se requiere de hardware adicional.
- En coma fija la posición de la coma fraccionaria hay que mantenerla por separado y ponerla en su lugar en el resultado al finalizar las operaciones. En coma flotante, la parte fraccionaria está incluida en el formato del número real.

La unidad Aritmético-Lógica

La CPU

Elemento fundamental del tratamiento de la información es la Unidad Central de Proceso, es un automata que opera en un ciclo infinito:

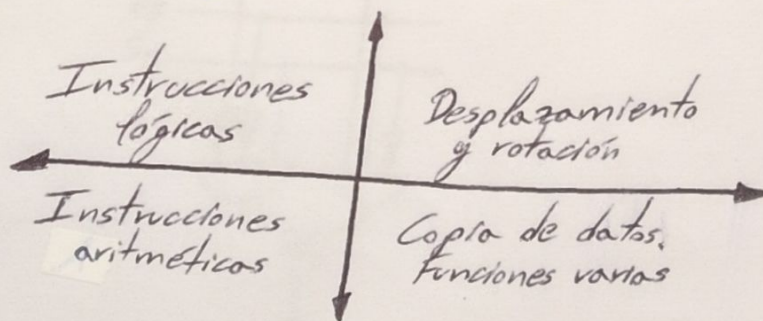
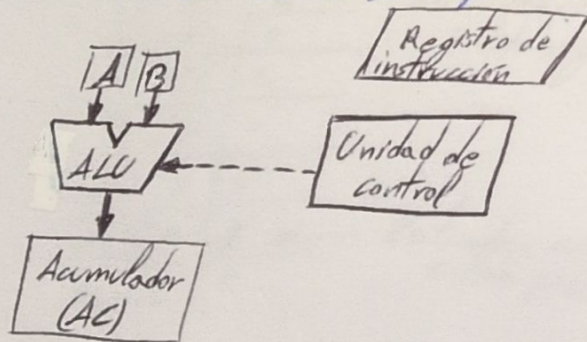
- Extracción de instrucciones de la memoria principal.
- Decodificar la instrucción.
- Obtener los operandos o datos que intervienen en la operación indicada.
- Procesar los operandos y producir un resultado.
- Escribir el resultado en la dirección que indica la instrucción.



La ALU

Realiza la transformación de los datos de entrada.

Es un procesador de propósito general sabe realizar operaciones lógicas y aritméticas, e instrucciones como la copia de datos entre registros o entre registros y memoria.



Registro de estados

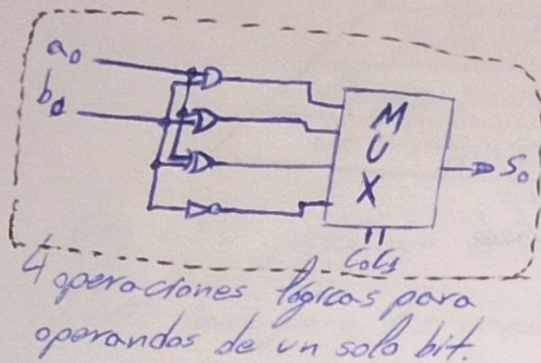
El registro de estados (flags de estados) consta de alrededor de 8 bits:

- S Flag de signo - Contiene el resultado de la última operación (positivo, 1 negativo).
- Z Flag de Ceros - "1" si el resultado es cero.
- C Flag de acarreo (carry).
- V Flag de overflow (desbordamiento).

Operaciones de la ALU

Lógicas

NOT AND NAND OR NOR XOR XNOR

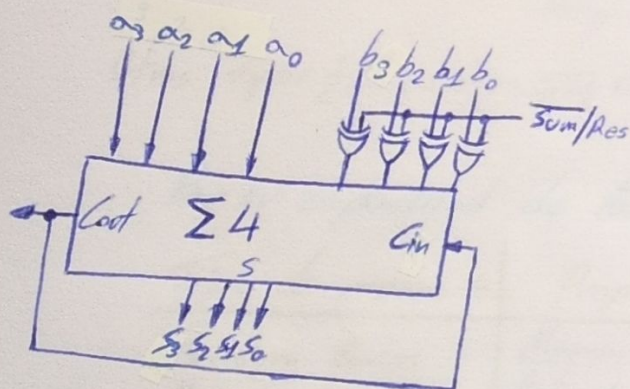


Aritméticas.

Existe el acarreo.

Sumador/Restador Ca1.

Sum/Res { 0 suma
1 resta



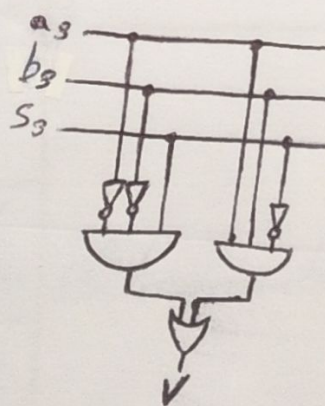
1. Se toman los operandos
2. Se realiza la suma (si era resta los XOR lo ponen en Ca1).
3. Se añade el acarreo al resultado
4. Se comprueba si overflow.

Overflow

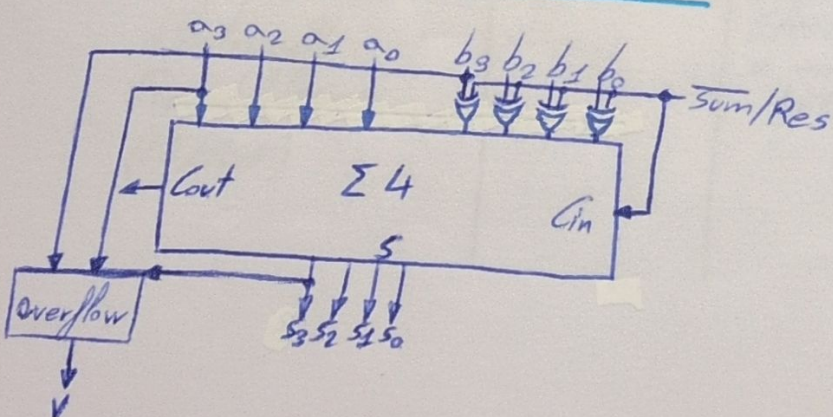
a3	b3	S3	Flag overflow
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	1	1
1	1	0	0

Entradas positivas salida negativa

Entradas negativas salida positiva

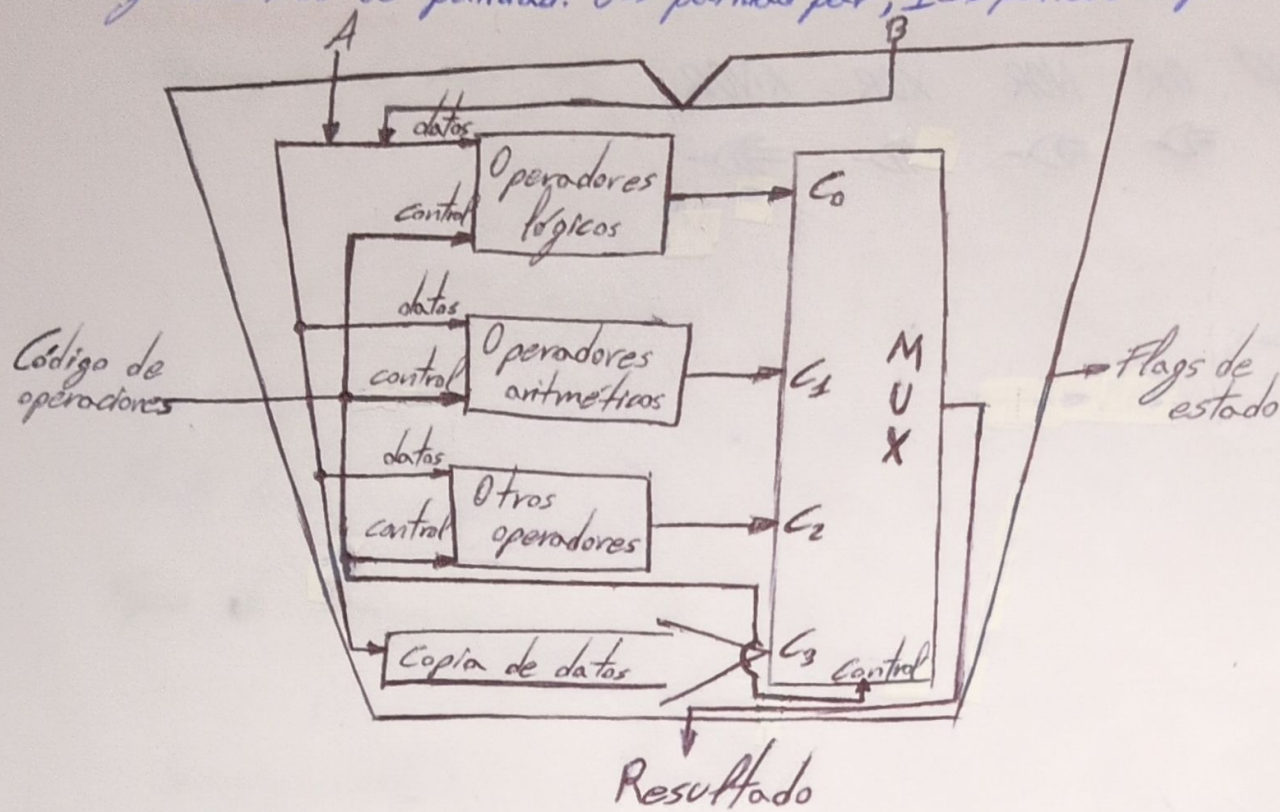


Sumador/Restador Ca2.



Diseño de la ALU.

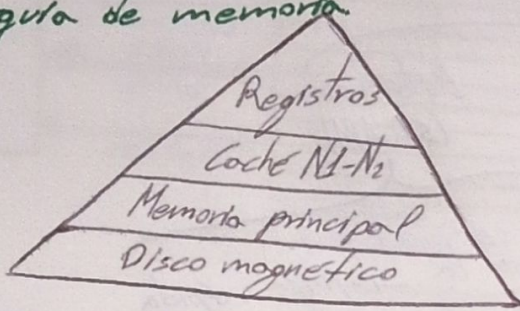
Flag de estado de paridad: 0 $\rightarrow$ paridad par, 1 $\rightarrow$ paridad impar.



La memoria

Características

Jerarquía de memoria



+Tiempo de acceso
 -Frecuencia de acceso.
 +Capacidad
 -coste por bit

La CPU junto con el sistema operativo se encargan del traslado entre memorias.
 Punto base para las memorias: el condensador $\rightarrow$ Bistables $\rightarrow$ Registros.

Tipos de memoria.

Semiconductores {

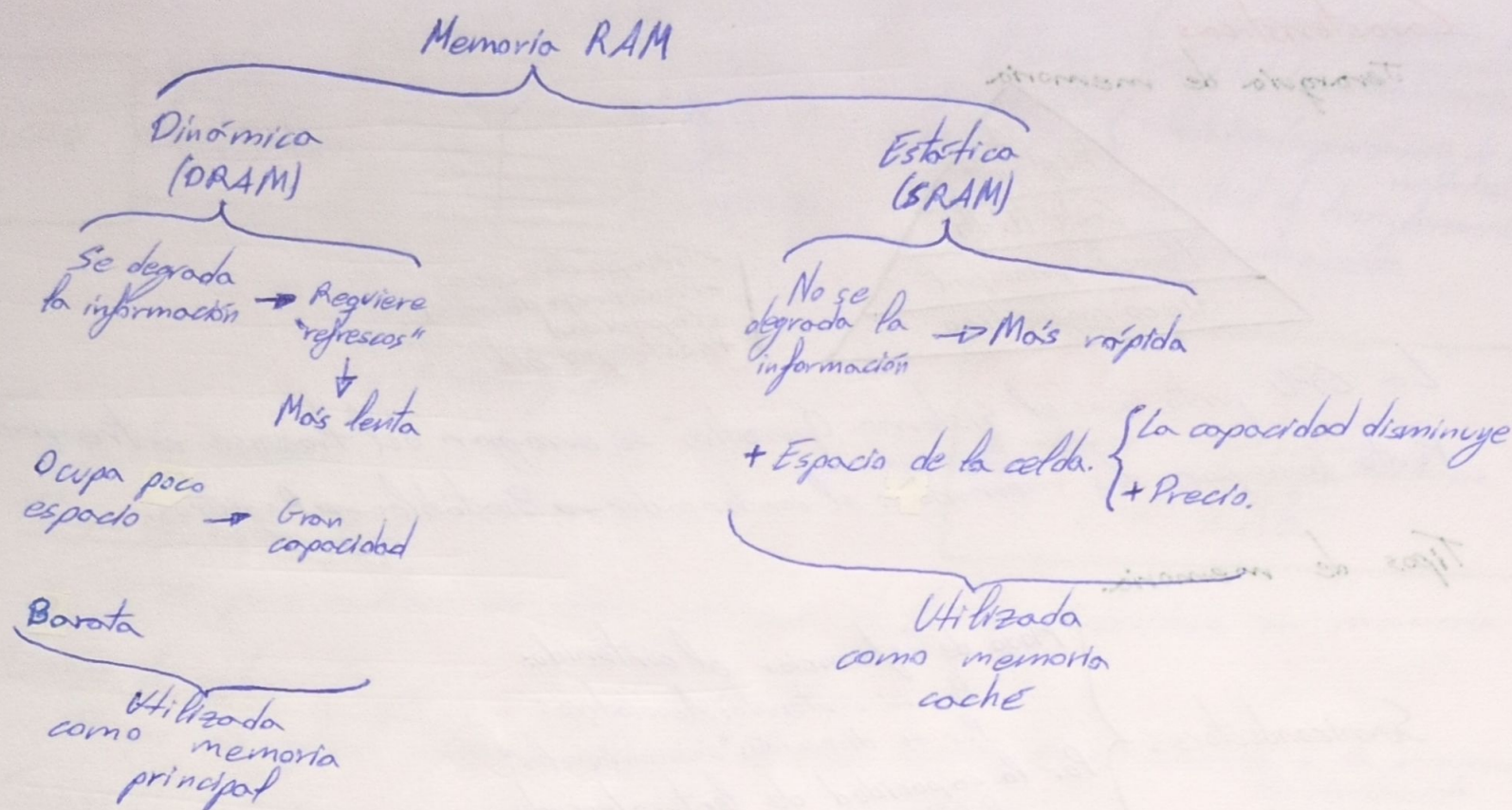
- Modo de referenciar el contenido:
 - Por su contenido: Asociativos
 - Por su dirección: "Convencionales"
- Por la capacidad de lectura/escritura:
 - RAM, ROM...
- Por su tecnología:
 - Estáticas
 - Dinámicas

Otros tipos { Disco duro, CD, DVD, cinta magnetica...

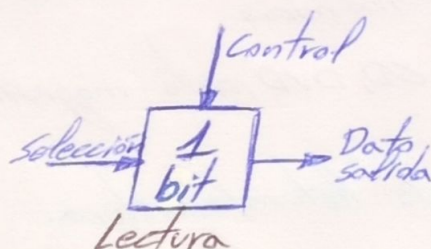
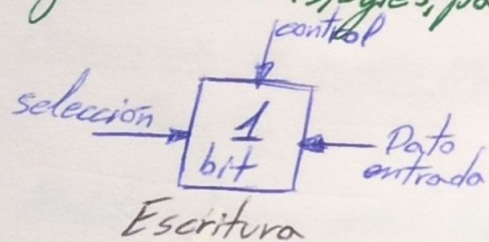
Por su capacidad de lectura/escritura.

Tipo de memoria	Propósito	Borrado	Modo de escritura	Volatilidad
Random Acces Memory (RAM).	Memoria de lectura/escritura	Electrico a nivel de byte.	Eléctricamente	Volátil
Read Only Memory (ROM)	Memoria de solo lectura.	No posible	Mascaras	No volátil
Programmable ROM (PROM)				
Erasable PROM (EPROM)		Luz UV a nivel chip	Eléctricamente	
Memoria Flash	Enfocados, sobre todo a lectura	Electrico a nivel bloque		
Electrically Erasable PROM (EEPROM)		Electrico a nivel byte		

Por su tecnología



Organización: bits, bytes, palabras



Se agrupan los bits en celdas. Cada celda tiene asignada una dirección, es la unidad direccionable de memoria.

Cada bit no tiene una dirección, cada celda sí.

La CPU no puede acceder directamente a un bit concreto de la memoria; una vez la celda está en algún registro de la CPU, esta puede acceder a cada bit según las instrucciones.

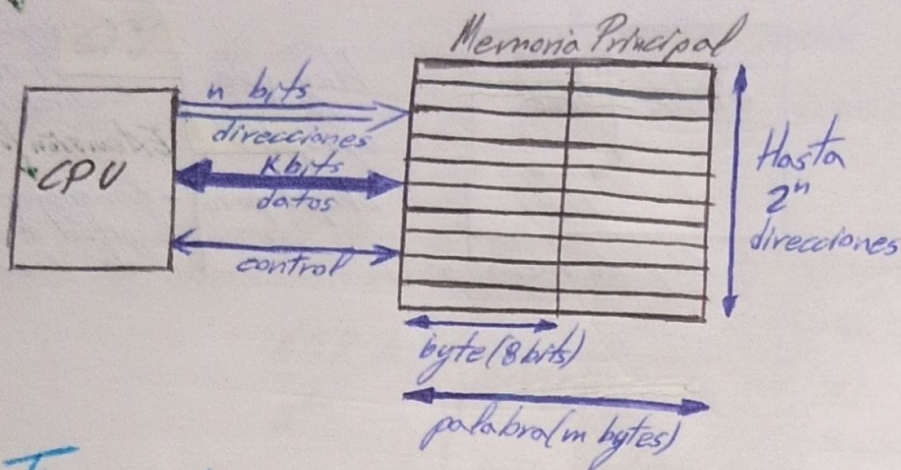
Los bits se ordenan en bytes comúnmente numerando de derecha a izquierda, siendo el más significativo el de la izquierda.

La ordenación de bytes se dividen en dos opciones comunes:

Big-endian → el byte más significativo del número en la dirección más baja de memoria.

Little-endian → el byte menos significativo del número se almacena en la dirección más baja.

Características de la memoria



Atener en cuenta sobre las memorias

- Capacidad + Espacio de direccionamiento
- Unidad direccionable $\rightarrow$ Dirección
- Palabra (organización de los módulos)
- Unidad de transferencia
- Tiempo de acceso

Puerta triestado

D	Z	CD	Z
0	x	Alta impedancia	
1	0	0	
1	1	1	

Tiempo de acceso

Tiempo de acceso (T_a)

Tiempo para realizar una única operación de lectura o escritura en memoria.

Tiempo de refresco (T_{ref})

Tiempo de refresco necesario después de cada acceso. $\rightarrow$ se recupera a la memoria su contenido

Tiempo de ciclo de memoria (T_c)

$T_c = T_a + T_{ref} \rightarrow$ Tiempo mínimo entre dos accesos.

Frecuencia de acceso (F_a)

$$F_a = \frac{1}{T_c}$$

Medidas de capacidad binaria

Bit $\rightarrow$ Unidad mínima de valores

Byte $\rightarrow$ 8 bits

Word $\rightarrow$ Logitud registro

Kilo - 2^{10}
 Mega - 2^{20}
 Giga - 2^{30}
 Tera - 2^{40}
 Peta - 2^{50}
 Exa - 2^{60}
 Zetta - 2^{70}
 Yotta - 2^{80}
 Bronto - 2^{90}

n dir $2K \times 8$ 2K direcciones
 m datos
 Número de celdas: $2K \rightarrow 2 \cdot 2$
 Tamaño de celda: 8 bits

$n = 11$ pines de direcciones
 $m = 8$ pines de datos.

C (capacidad de almacenamiento)

1K8 Byte
 1K6 bit

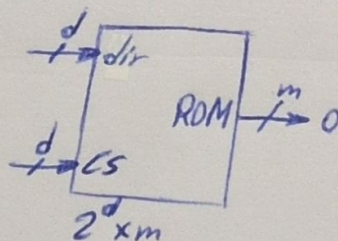
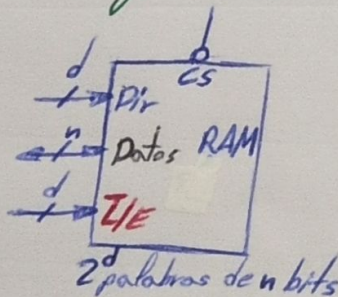
$$C = 2^n \times m$$

$$C = 2^{11} \times 8 = 2^{11} \times 2^3 = 2^{14}$$

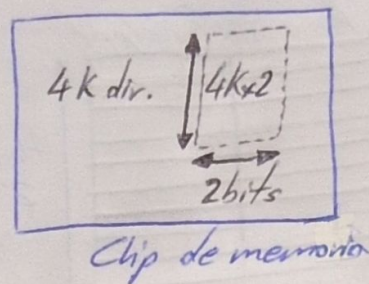
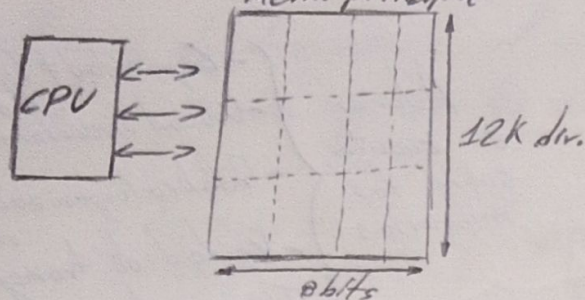
$$C = 2^{14} = 16384 \text{ bits}$$

La memoria principal

Interfaz de una memoria



Organización de los módulos



Extensión del nº de palabras.
- Conseguir el espacio de direccionamiento
Extensión longitud palabra
- Conseguir la longitud de la celda de memoria

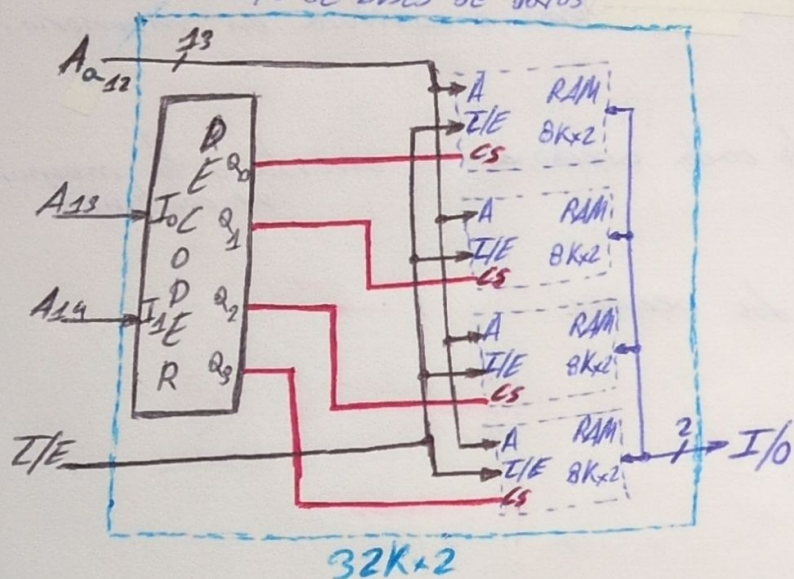
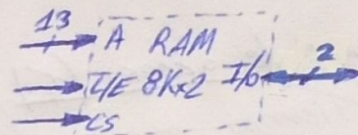
Hag que agrupar chips para

Ampliación del nº de palabras

Se quiere un 32Kx2 con RAM's de 8Kx2

$$8K \times 2 = 2^3 \cdot 2^{10} = 2^{13} \rightarrow 13 \text{ bits de bus de direcciones}$$

2 bits de buses de datos

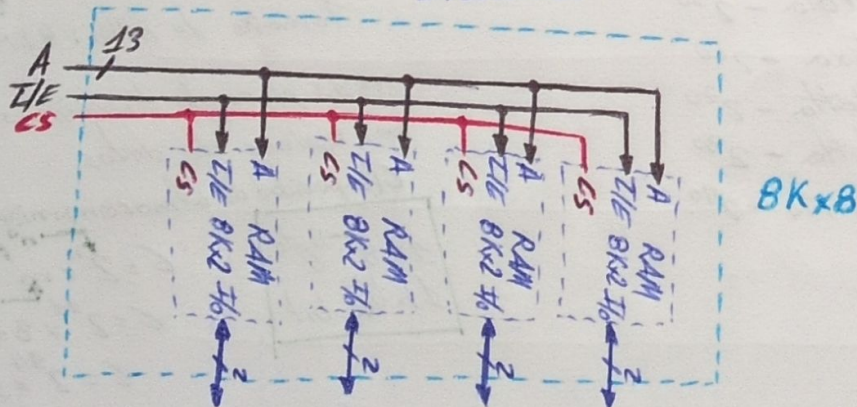


Ampliación de la longitud de palabra.

Se quiere un 8Kx8 con RAM's 8Kx2.

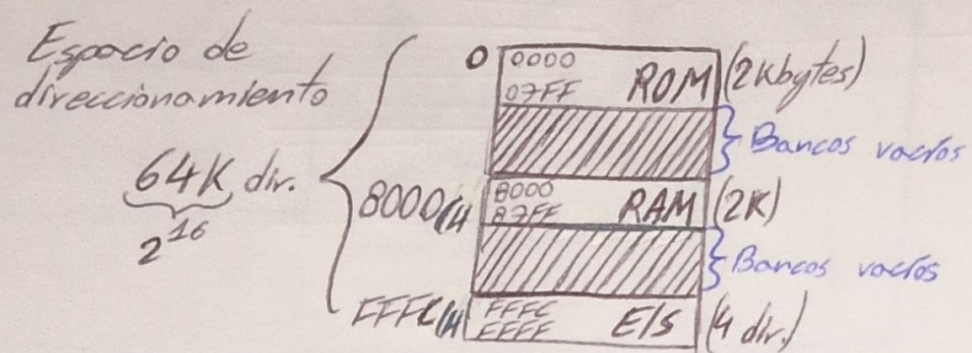
$$2^3 \cdot 2^{10} = 2^{13} \rightarrow 13 \text{ bits de bus de direcciones}$$

2 bits de buses de datos

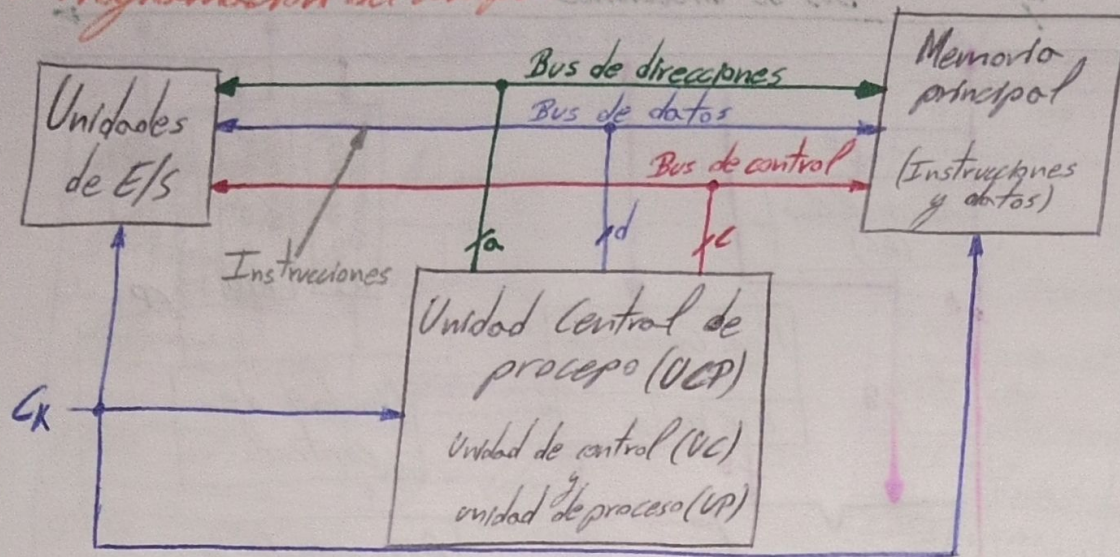


El mapa de memoria

Es el área de direcciones al cual se puede acceder, el mapa se subdivide en bancos de memoria y se les asignan direcciones de memoria.



Tema 5 Programación del computador



Instrucción: Cada orden diferente que realiza el computador.

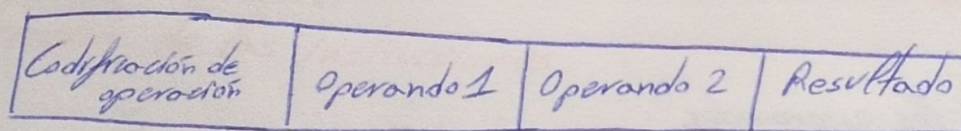
Juego de instrucciones: Conjunto de instrucciones que ejecuta un computador.

Programa: Secuencia ordenada de instrucciones.

Fases de ejecución de una instrucción

- Mientras no se indique lo contrario en cada fase dura un ciclo de reloj.
- Extracción de la instrucción de la dirección de memoria indicada por el Contador de Programa (CP) (fetch).
 - Decodificación de la instrucción. (Operación a realizar).
 - Búsqueda de operandos.
 - Ejecución (operación en la ALU).
 - Almacenamiento de los resultados (registros generales o dirección de memoria principal).
 - Interrupción (terminada la ejecución de la instrucción y se comprueba si ha habido interrupción).

Formato



Modos de direccionamiento (Donde se ubican los operandos).

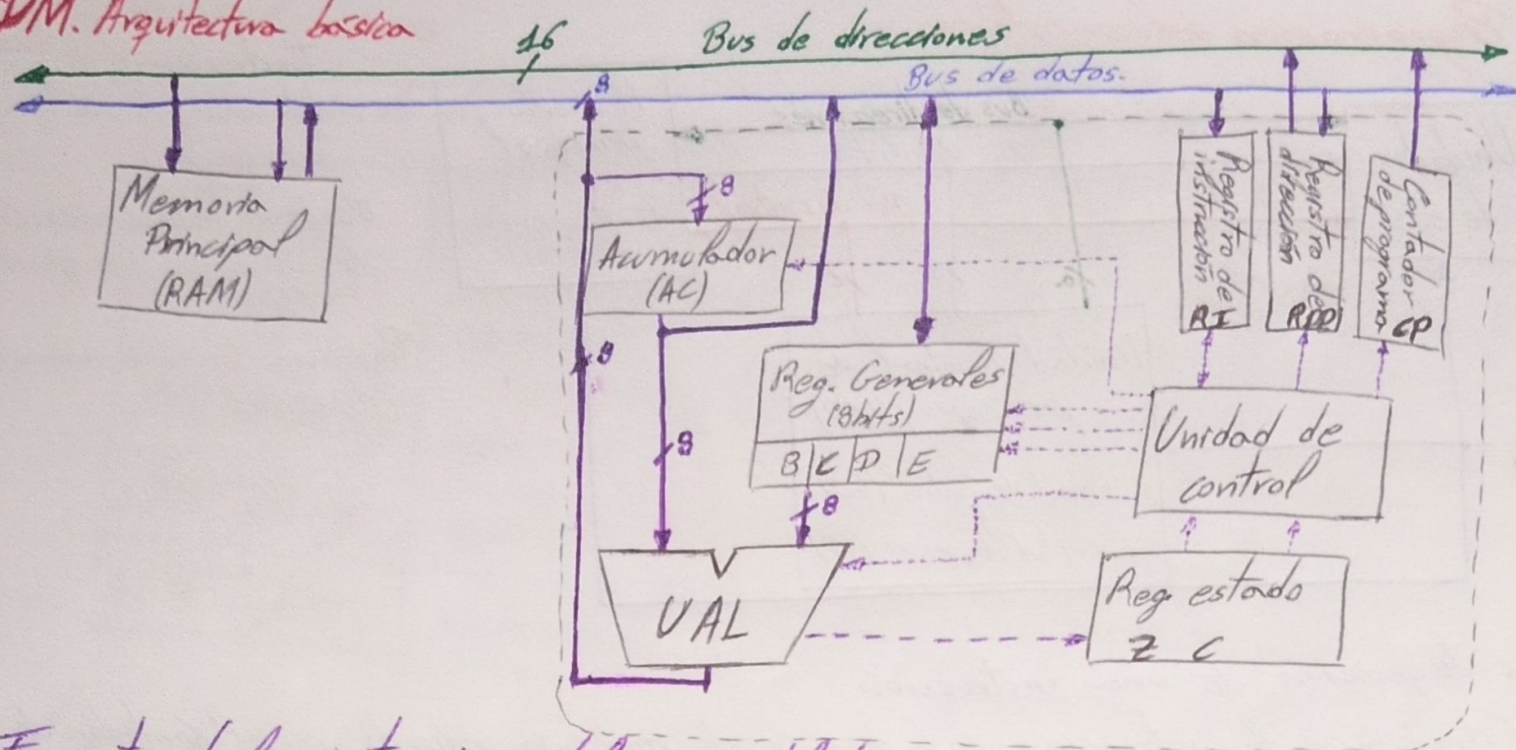
- | | | | |
|-------|------|-----------|--|
| C.op. | Opad | Modo | Descripción |
| | | Immediato | En la instrucción instrucción |
| | | Directo | En la dirección |
| | | Indirecto | En la dirección de otra dirección |
| | | Implícito | Contenido en fases de instrucción |
- Detalles de direccionamiento:
- Immediato:**
 - Valor
 - Dirección { - memoria del valor, - registro
 - Directo:**
 - Absoluto { - A registro, - A memoria
 - Relativo { - Al CP, - A registro
 - Indirecto:**
 - Absoluto { - A registro, - A memoria
 - Relativo

Lenguaje ensamblador

- Cada línea de programa es el nombre de una instrucción.
- Correspondencia directa y biunívoca entre cada instrucción de ensamblador y la correspondiente instrucción en código máquina.
- El programa generado en este lenguaje se dice que está "desensamblado".

El ensamblador depende del tipo de máquina que se utilice. El ensamblador de un micro es distinto a otro según la estructura interna del micro.

PDM. Arquitectura básica



Formato de las instrucciones del ensamblador.

Etiqueta:

- Primer campo de la instrucción
- Identifica la línea.
- Suele usarse para saltos.
- Tiene máximo de 8 a 10 caracteres.
- Se debe colocar la primera letra de la etiqueta en el primer carácter de la línea.
- El primer carácter debe ser alfabético.

Código:

- El segundo campo es el del nemónico de la instrucción. Representa la operación a realizar.

Operandos:

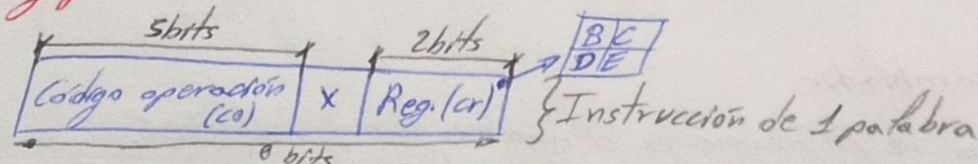
- El tercer campo identifica los operandos.
- Indica la dirección física (registro o posición de memoria), o el dato programáticamente dicho.
- En las direcciones que usan direccionamiento implícito, este campo no existe.
- Cuando hay dos operandos, éstos se separan por comas; el primero de ellos es el operando destino.
- Pueden ser expresiones o constantes (numéricas o alfabéticas), con diferentes notaciones para binario (B o %), octal (O, @, Co o Q), hexadecimal (H o \$) y decimal (D o sin especificador).

Comentarios:

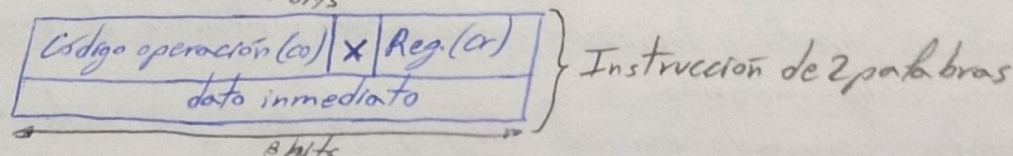
- Campo opcional
- Van precedidos de ";"

Modos de direccionamiento y formato

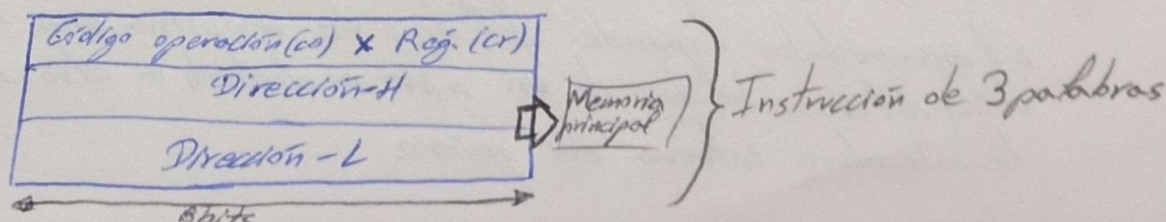
Para direccionamiento a registro



Para direccionamiento inmediato



Para direccionamiento a memoria



Juego de Instrucciones

Transferencia de datos

^{load} LD Reg	Acc ← reg
^{store} ST reg	reg ← Acc
^{load immediate} LDI num, reg	reg ← num
^{load memory} LDM dir, reg	reg ← (dir)
^{store memory} STM reg, dir	(dir) ← reg

Aritméticas

ADD reg	$Acc \leftarrow Acc + reg$
SUB reg subtract	$Acc \leftarrow Acc - reg$
CMP reg compare	$Acc - reg$ flags(Z, C)
INC Increment	$Acc \leftarrow Acc + 1$
ADI reg Add immediate	$Acc \leftarrow Acc + num$
SUI reg Subtract immediate	$Acc \leftarrow Acc - num$
CMI num compare immediate	$Acc - num$ flags(Z, C)
LF load flags	$Acc \leftarrow FLxxxxxxFZ$

$$A_c = X \{ FZ=1$$

$$A_c \geq X \{ FC=1$$

$$A_c < X \{ FZ=FC=0$$

maior
significativa

Logreans $\leftarrow$ No afectan los flaps de estado.

ANA R	Ac Ac AND reg
ORA R	Ac Ac OR reg
XRA R	Ac Ac XOR reg
CMA complement Ac	Ac NOT(Ac)
ANI num	Ac Ac AND num
ORI num	Ac Ac OR num
XRI num	Ac Ac XOR num

Instrucciones de salto

JMP dir	CP ₄ dir
BEQ dir Branch on Equal	Si $FZ_{eq} = 1 \Rightarrow CP_{4} \text{ dir}$
BC dir Branch on Carry	Si $FC_{arry} = 1 \Rightarrow CP_{4} \text{ dir}$

Satto
incondicional

En la operación $A-B$
con números naturales si $A > B \Rightarrow A \text{ con } B$

Codificación de las instrucciones

	Transferencia				Aritméticas				Lógicas				Salto					
CO ₄	0				0				1				1					
CO ₃	0				1				0				1					
Bytes	10'2		3		1		2		3		2		3					
CO ₂	0		1		0		1		0		1		0					
CO ₁	0	1	0	1	0	1	X	0	1	0	1	0	1	0	1			
CO ₀	0	1	X	X	X	0	1	0	1	X	0	1	0	1	X	X	0	1
LD reg																		
ST reg																		
LDI num reg																		
LDM dir reg																		
STM reg dir																		
ADD reg																		
SUB reg																		
CMP reg																		
INC																		
LF																		
ADI num																		
SUI num																		
CMI num																		
ANA reg																		
ORA reg																		
XRA reg																		
CMA																		
ADI num																		
ORI num																		
XRI num																		
JMP dir																		
BEQ dir																		
BC dir																		

CO ₄	CO ₃	CO ₂	CO ₁	CO ₀	X Reg
-----------------	-----------------	-----------------	-----------------	-----------------	-------

Código de operación

- Si se hace referencia a un solo registro o 1 palabra (1 Byte; 8 bits)
- Si se hace referencia a variables (NUM, con la I (Immediate)) o 2 palabras (2 Bytes).
- Si se hace referencia a direcciones (instrucción, parte alta dir, parte baja dir.) o 3 palabras.

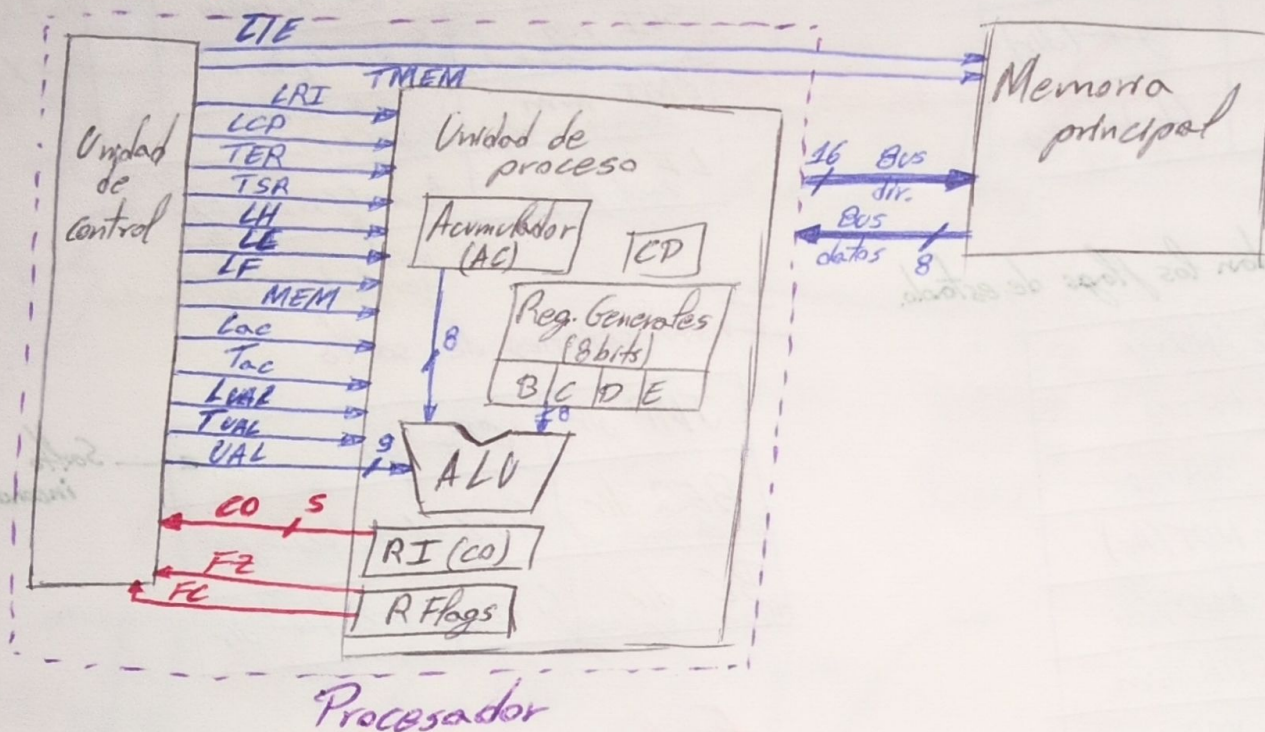
Codificación de las registras

CR ₁	CR ₀	Registro
0	0	B
0	1	C
1	0	D
1	1	E

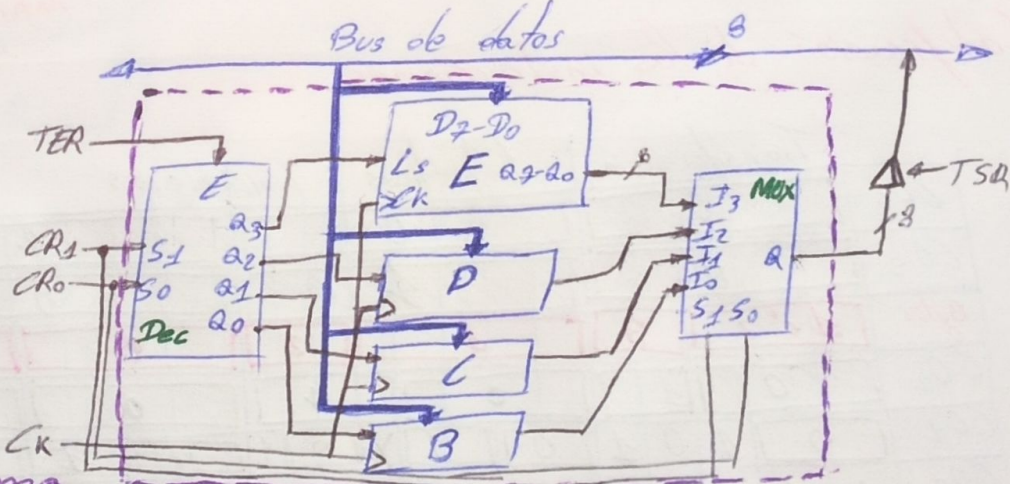
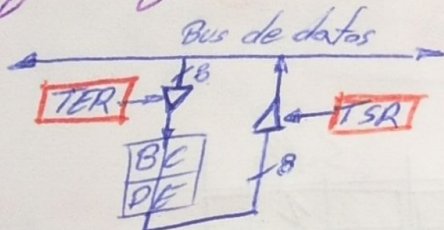
1 palabra
1 Byte
8 bits

Código operación	X	CR ₁	CR ₀
------------------	---	-----------------	-----------------

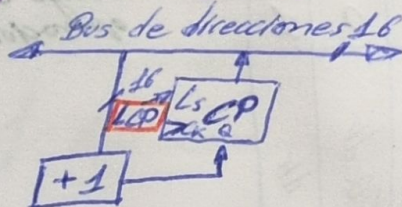
Tema 6 La Unidad de control



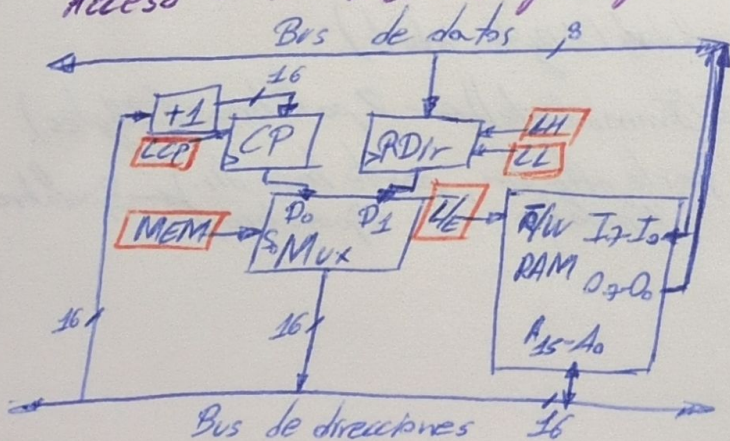
Registros generales



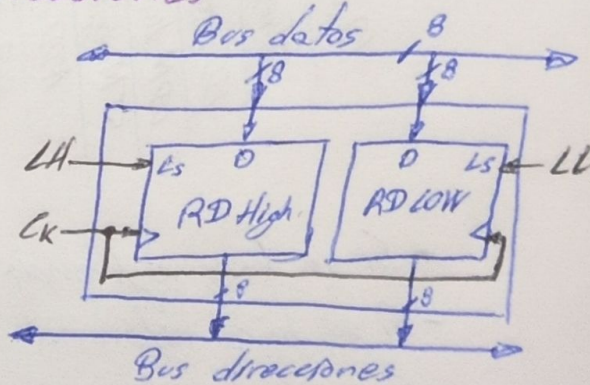
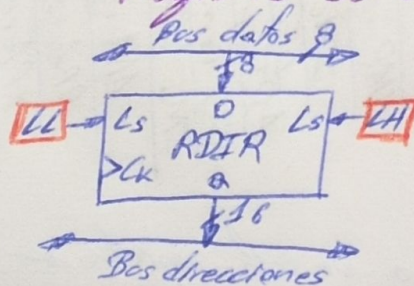
Registro de contador de programa

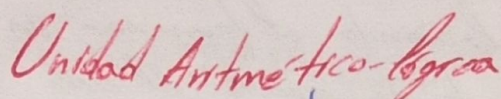
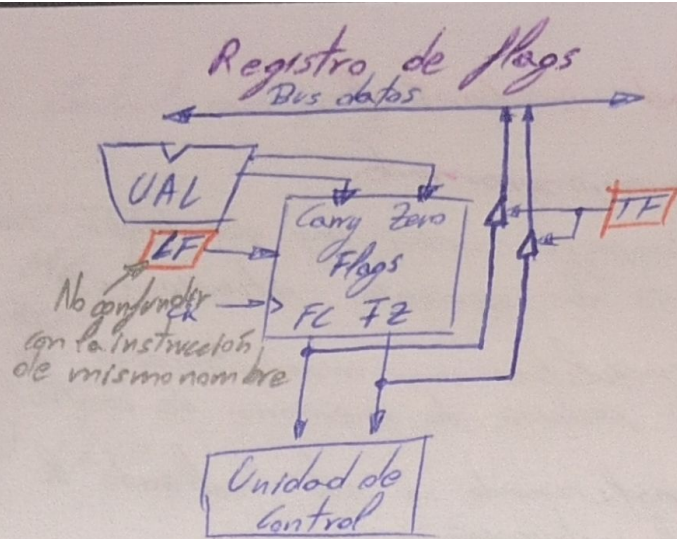


Acceso a la memoria principal

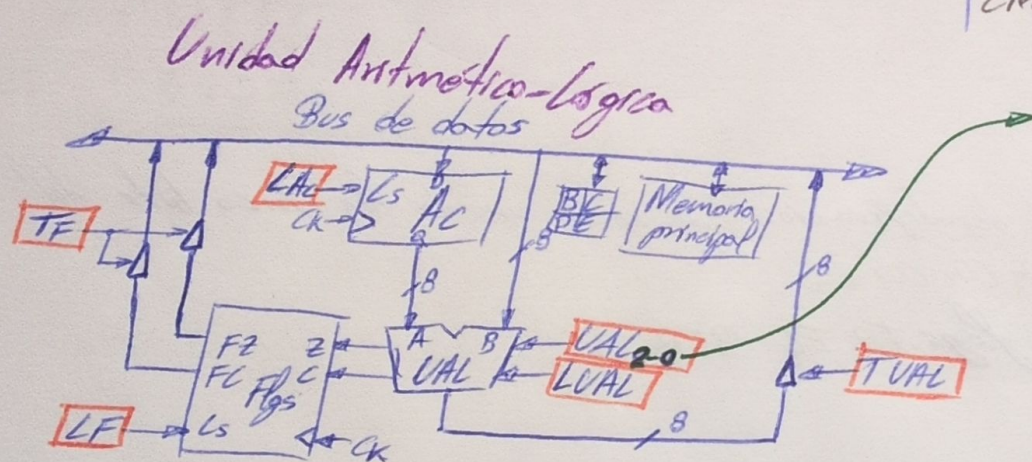


Registro de direcciones





	ADD reg ADI num	Sumador	Ac	Flags
Aritmetica	INC	Sumador + 1	Ac	Flags
	SUB reg SUI num	Restador	Ac	Flags
	CMP reg CMI num	Restador	X	Flags
	ANA reg ANI num	AND	Ac	X
Logica	ORA reg ORI num	OR	Ac	X
	XRA reg XRI num	$\oplus$	Ac	X
	CMA	NOT	Ac	X



Operation	VAL_1	VAL_2	VAL_0
$A+B (Caz)$	0	0	0
$A-B (Caz)$	0	0	1
$A \text{ and } B$	0	1	0
$A \text{ or } B$	0	1	1
$A \oplus B$	1	0	0
$\text{not } A$	1	0	1
Librefeserva	1	1	0
$A+1$	1	1	1

Fases de ejecución

- 1.- Extracción de la instrucción (Fetch).
- 2.- Decodificar la instrucción. (Decode).
- 3.- Obtener operandos. (Operand).
- 4.- Ejecución (UAL) (Execution).
- 5.- Almacenamiento del resultado (Write).

Diseño de la Unidad de Control

Unidad de control microprogramada

Al realizar la microprogramación hay que tener en cuenta que nos encontraremos ante las siguientes situaciones:

- Ejecución secuencial: tras una microinstrucción se ejecuta la almacenada en la siguiente posición de memoria de control (MC).
- Salto condicional: cuando se esté realizando la decodificación, en función del código de la instrucción.
- Salto incondicional: cuando al terminar un microprograma se tiene que ir a la dirección donde se ejecuta el ciclo de Fetch de la siguiente microinstrucción.

Dos campos dentro de la microinstrucción resuelven esto:

Tabla de secueñciamiento

- SALTO que indica la dirección de la MC a la que debe saltar.
- TEST donde se indica que condición debe cumplirse el salto.

• El número de direcciones de la MC es de 64, el número de entradas de dirección de la MC será de 6.

• Condiciones de test.

- 9 tipos de saltos
- Salto condicional según la decodificación de cada uno de los cinco bits del código de operación. $CO_4 CO_3 CO_2 CO_1 CO_0$
 - Salto condicional según los flags. FZ FC BEQ BC
 - Salto incondicional. JMP
 - Ejecución secuencial. Salto de instrucción en instrucción

• Habrá que evaluar nueve condiciones por lo que necesitaremos 4 bits en el campo de test

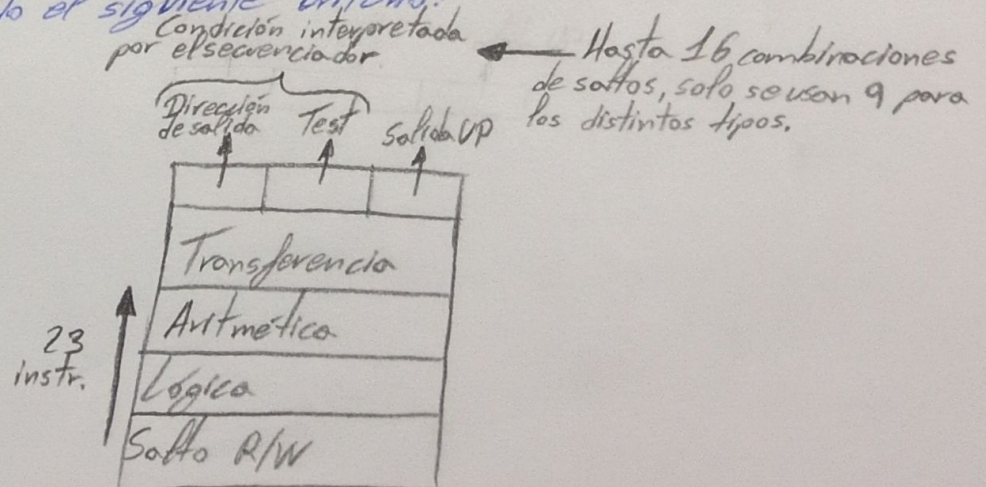
• El número de señales de control es de 18.

La capacidad de la MC es de 64 palabras de 28 $(6+4+18)$ bits.

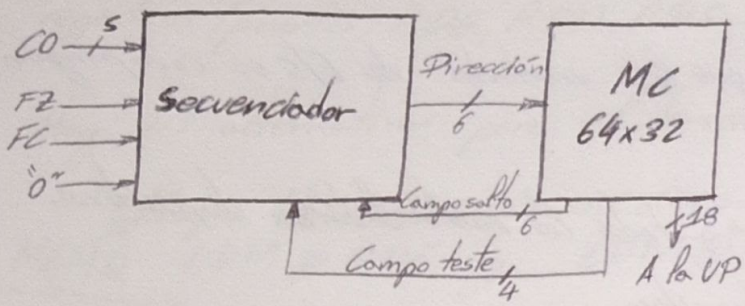
$$C_{MC} = 2^6 \times 28$$

Para el secuenciamiento del código de operación y de los flags, se utiliza un Multiplexor de ocho canales, teniendo el siguiente aspecto:

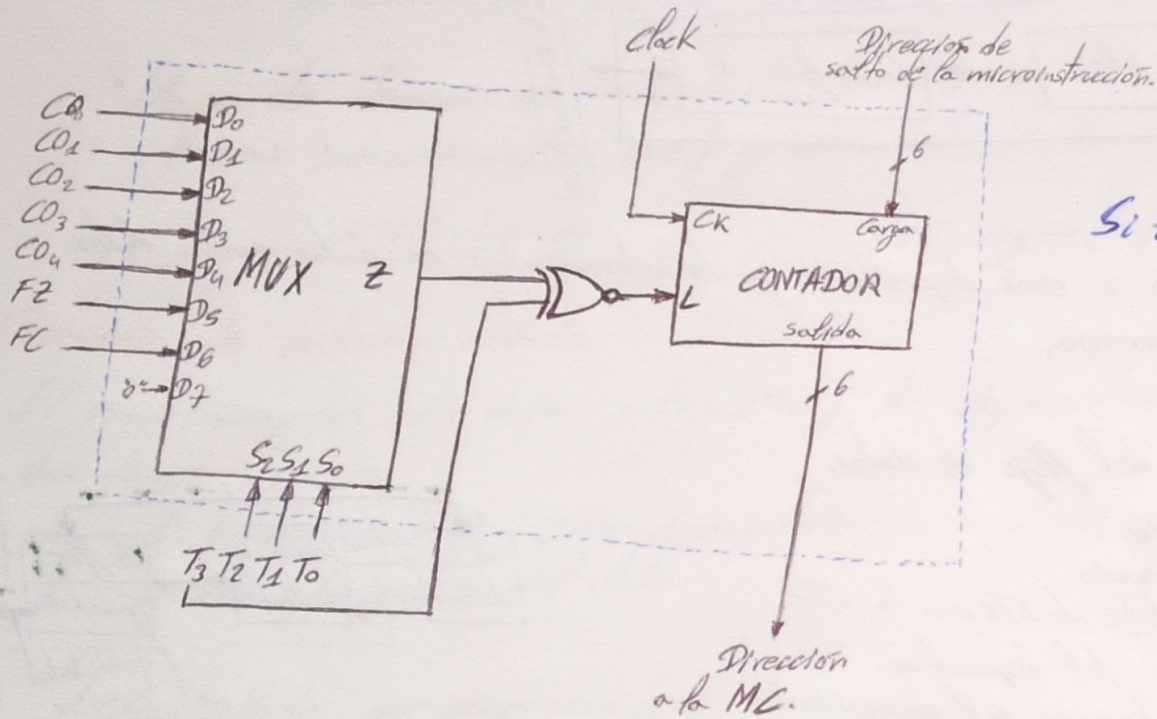
T_2	T_1	T_0	Condición
0	0	0	CO_0
0	0	1	CO_1
0	1	0	CO_2
0	1	1	CO_3
1	0	0	CO_4
1	0	1	FZ
1	1	0	FC
1	1	1	"0"



Esquema de la Unidad de Control microprogramada



Secuenciador

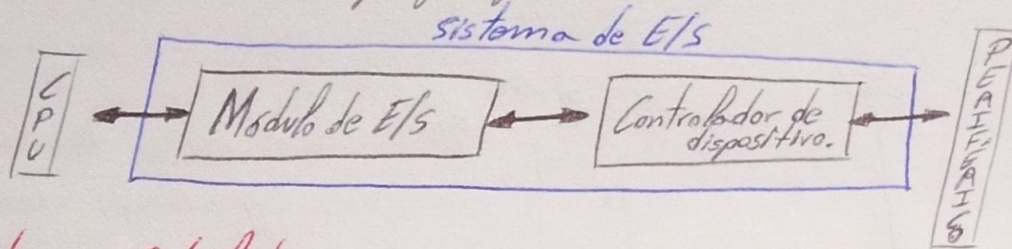


Tema 7 Sistemas de Entrada/Salida

Estructura de un sistema de E/S.

La gran diversidad de dispositivos periféricos hace que las unidades de E/S se compongan de dos partes:

- **Módulo de E/S:** Soporta las características "comunes" a todos los ~~controladores~~ dispositivos.
- **Controlador del dispositivo:** Específico para cada dispositivo.

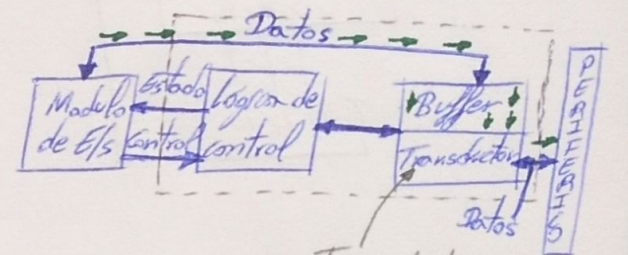


Cometido de un controlador

Responsable del control de uno o más dispositivos y del intercambio de datos entre tales dispositivos y la CPU, o la memoria.

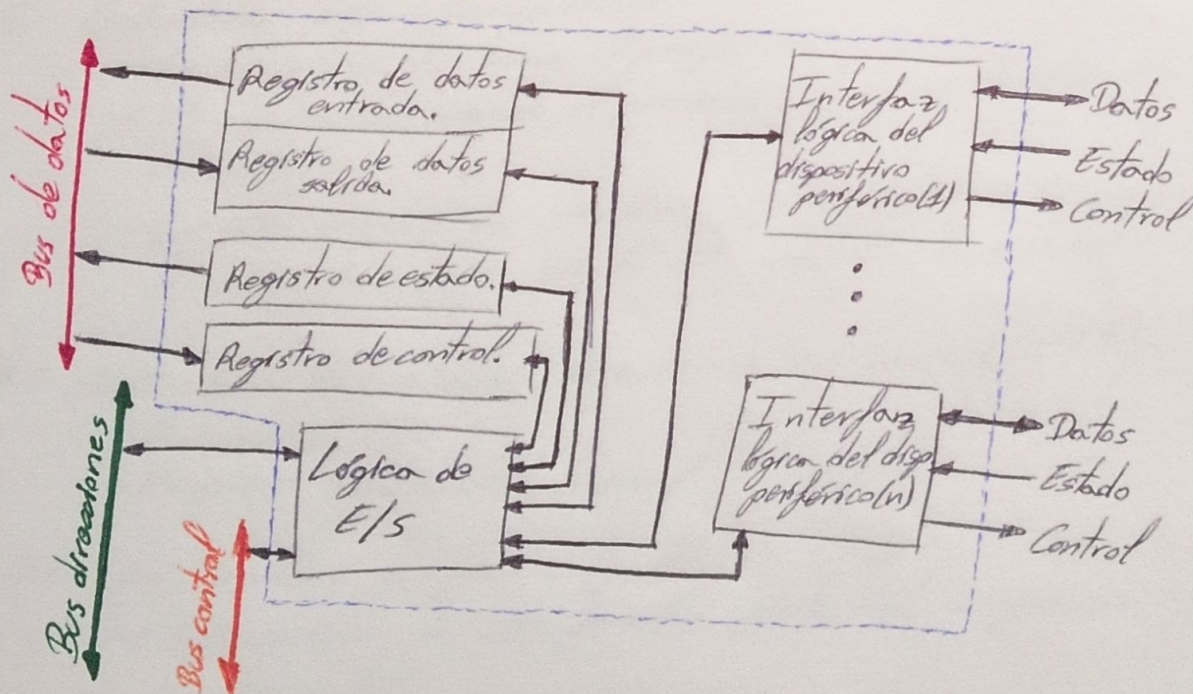
Funciones:

- Control y temporización del flujo de datos.
- Comunicación con la CPU.
 - Decodificación de comandos.
 - Intercambio de los datos de E/S con la CPU.
 - Informar del estado del dispositivo.
 - Reconocimiento de la dirección del dispositivo.
- Comunicación con el dispositivo.
- Almacenamiento temporal de datos (buffer).
- Detección de errores.



Transductor: conversor de necesidades eléctricas a la naturaleza del dispositivo.

Estructura de un controlador



Registros de Datos de Salida (RDS) } Registros de datos

Registros de Datos de Entrada (RDE) }

Registro de Control de Dispositivo (RCD) - Se le proporcionan los ordenes.

Registro de Estado del Dispositivo (RSD) - Responde a la CPU.

Lógica de E/S - Proporciona la activación del controlador y responsable entre los registros y el periférico(s).

Interfaz lógica del dispositivo - Soporta el intercambio de datos, control y estado.

Acceso a los dispositivos de E/S.

Para acceder a los bloques funcionales de E/S, la CPU direcciona cada uno de los registros (puertas: RDE, RSD, RDS, RCD) que conforman los controladores.

Hay dos alternativas para el direccionamiento de los dispositivos de E/S:

- Mapa de memoria común: se comparte el espacio de direcciones de memoria.
MEMQ LDM @ RDE, reg LDM @ RSD, reg STM reg, @ RDS STM reg, @ RCD.
mapa de memoria normal (LDM, STM)
Cada periférico ocupa 4 direcciones del mapa de memoria, estas direcciones son denominadas puertos de E/S.

- Mapa de direcciones independientes. Espacio de direcciones propio.
Nuevas instrucciones (IN, OUT) y nueva entrada de control ($\overline{IORA}$)

IN(dir) $\rightarrow$ reg

OUT reg $\rightarrow$ (dir)

Técnicas de entrada salida.

El envío/recepción de datos entre CPU y los dispositivos de E/S se realiza en dos fases:

1. Sincronización CPU-dispositivo.
2. Transferencia del dato.

Tipos:

1.- Controlada por programa (polling)

- La iniciativa de transferir la toma la CPU.
- E/S sincronas con la ejecución del programa, cuando el programa lo permite. El programador decide en qué momento ocurre.
- No requiere hardware adicional.

2.- Controlada por interrupciones.

- La iniciativa la toma el periférico.
- E/S asincronas, cuando el periférico lo desea. Pueden tener lugar en cualquier momento.
- La CPU dispone de una línea especial **IRQ** (Interrupt Request) que activan los periféricos para que la CPU atienda la solicitud.

3.- Controlada por acceso directo a memoria (DMA).

- Iniciativa CPU.
- Requiere hardware adicional

E/S por sondeo o programa (polling)

- Las instrucciones de E/S forman parte del programa que se está ejecutando en el procesador.
- El procesador deja la tarea que está realizando y ejecuta un programa (rutina) que controla directamente la operación de E/S.

La CPU está supeditada a la ejecución del programa (tiempo de CPU total).

E/S por interrupción

Interrupción

- Suceso más o menos esperado que no se conoce el momento en el que va a suceder.
- Se debe atender lo antes posible.
- Puede solicitarla el dispositivo o la CPU.
- Se abandona el flujo secuencial de ejecución del programa.
- Dar control a la Rutina de Tratamiento de Interrupción (RTI).
- Se ejecuta la RTI.
- Devolver el control al punto del programa principal donde se produjo la interrupción.
- Se debe guardar el contexto del programa en ejecución (estado de la CPU en el momento de la interrupción) en la pila o STACK.

La pila es una zona de memoria común a todos los programas donde se va "apilando" la información, lo que exige que se lea en sentido contrario en que se escribe. Se direcciona por medio de un registro de propósito especial, el Stack pointer - SP - (contador UP/DOWN).

↑ Se salva en la "pila" la dirección de retorno y el contenido de los flags.

La velocidad de transferencia está limitada por la velocidad de trabajo de la CPU.

Interrupciones enmascarables: Aquellas que a voluntad del programador pueden ser permitidas o no permitidas. El procedimiento utilizado es mediante la activación del flag del registro de estado de la CPU.

Interrupciones no enmascarables: (NMI) Siempre se atienden

Lo habitual es que la CPU disponga de una sola entrada de **IRQ** compartida por todos los dispositivos periféricos. Para identificar la fuente de interrupción:

Controlador Programable de Interrupciones (PIC).

- Controla a varios controladores. Maneja lógica de prioridad.
- Solicita interrupción a través de una sola entrada "INTR".
- Reconoce y asigna la interrupción a cada uno de los periféricos. La CPU obtiene del PIC toda la información del periférico que interrumpió.
- A través del vector de interrupción, da la información necesaria para que el procesador asigne la rutina de interrupción correspondiente.
- Permite gestionar prioridades y métodos de enmascaramiento.

Enmascaramiento selectivo: Cada periférico tiene asignado un "bit de máscara", que permitirá a la CPU habilitar o deshabilitar su línea de interrupciones a nivel individual.

Enmascaramiento por nivel: Solo se permitirán interrupciones de aquellos periféricos cuyo nivel sea mayor que el nivel de referencia fijado por el programador en el Registro de Nivel.

Aceptación

- 1.- El procesador no acepta interrupciones mientras está ejecutando una instrucción, excepto RESET y Bus Error.
- 2.- Al finalizar la ejecución de la instrucción, comprueba si hay una interrupción NMI, si la hay, la atiende.
- 3.- Comprueba si en el registro de estado están permitidas o inhibidas las interrupciones. Si están inhibidas, continúa con la ejecución del programa. Si están permitidas verifica si está activada INTR, si es así, comienza el tratamiento de la interrupción.

Tratamiento

- 1.- El registro de estado y el contador de programa a la pila.
- 2.- Se inhiben las interrupciones.
- 3.- Activar el reconocimiento de interrupción.
- 4.- El controlador programable de interrupciones (PIC) desactiva la señal de interrupción (INTR).
- 5.- El PIC coloca el número del vector de interrupción en el bus de datos.
- 6.- La CPU lee por el bus de datos el vector, consigue la dir. del vector y pone la dir. en el contador de programa.
- 7.- Toma control la RTI correspondiente.
- 8.- Al finalizar RTI devuelve el control al programa principal y continúa la ejecución.

E/S por Acceso Directo a Memoria (DMA)

Transferencia rápida, memoria y periféricos, sin intervención de la CPU, de bloques de datos.

- Controlador DMA: Dirige la memoria en el proceso de transferencia.



- La CPU cede el control de los buses a la CDMA
o no inmediato

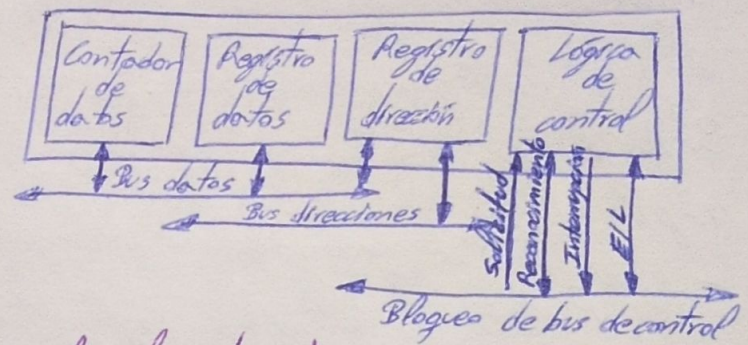
- El CDMA se inicializa mediante la ejecución de una rutina de inicialización formada por un conjunto de instrucciones.
- Al finalizar la operación de E/S también ejecuta una rutina de finalización para avisar que la transferencia ha finalizado.

Controlador Acceso Directo a Memoria

Cuando el procesador desea transferir un bloque de datos, envía una orden al módulo DMA para delegar la operación E/S, indicando:

- Lectura o escritura (bus de control).
- Posición inicial de memoria para la lectura o almacenamiento del bloque de datos (bus datos). Se almacena en "Registro de dirección". Se incrementa automáticamente después de cada transferencia.
- Capacidad de bloque; n° de palabras (bus de datos). Se almacena en Registro de cuenta de datos "Contador de datos". Se decrementa después de cada transmisión.
- "Lógica de control". Cuando el contador de datos llega a 0 indica que se ha finalizado la transmisión.

Luego el procesador continúa con su actividad.



Tipos:

Robo de buses.

Transferencia de bloque (normalmente 1 byte (si no se especifica lo contrario)).

El CDMA realiza transferencias de bloques completos de información cuando toma el control de los buses.

- Alta velocidad de transferencia de información.
- El procesador puede sufrir largos periodos de inactividad. (CPU incommunicada).
- Se realiza en plena ejecución del programa.

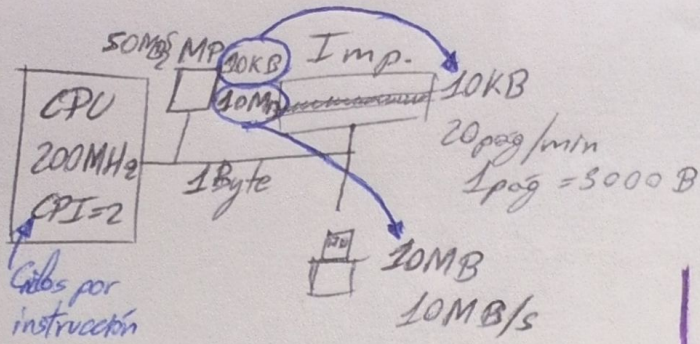
Robo de ciclo → Transferencia por ráfagas

El CDMA toma control de los buses cuando desea transferir datos solo durante un ciclo de CPU luego cede al CPU los buses hasta robar otro ciclo.

- Puede haber robos de ciclos durante mucho tiempo o robar más de un ciclo.

Bus transparente

El CDMA toma el control de los buses únicamente cuando la CPU está ejecutando fases de instrucciones que no hacen uso de dichos buses (decodificación, evaluación de los flags...).



Dependencia del bus → E/S por programa → CPU ocupada todo lo que dura el periférico

Impresora $V = \frac{\text{Capacidad}}{t} \Rightarrow t_{\text{CPU}} = \frac{C}{V} = \frac{10\text{KB}}{1\text{KB/s}} = 10\text{seg}$

$$\frac{20\text{pag}}{1\text{min}} \cdot \frac{3000\text{B}}{1\text{pag}} \cdot \frac{1\text{min}}{60\text{seg}} = 1\text{KB/s}$$

Disco de almacenamiento

E/S por programa

$$t_{\text{CPU}} = \frac{10\text{MB}}{10\text{MB/s}} = 1\text{seg}$$

E/S por interrupción

$$\begin{aligned} \text{nº interrupciones} &= \frac{1\text{ interrupción} \cdot 10 \cdot 10^6\text{B}}{1\text{Byte}} \\ &= 10^7 \text{ interrupciones} \\ \text{nº instrucciones} &= 10 \cdot 10^7 = 10^8 \text{ instr.} \end{aligned}$$

$$t_{\text{CPU}} = \text{nº instr.} \cdot t_{\text{instr.}} \quad \text{RTI (1 interrupción) 10 instr.}$$

10ns

$$t_{\text{CPU}} = 10^8 \cdot 10 \cdot 10^{-9} = 1\text{seg.}$$

instrucciones de interrupción

E/S por interrupción → La CPU apunta donde se para y atiende la tps. y los flags. rutina de interrupción.

rutina de tratamiento por instrucción de interrupción. $\text{RTI} = 10\text{instr.} + \text{dato}$

$10\text{KB} = 10.000\text{B} = 10.000 \text{ interrupciones.}$

$$t_{\text{instr.}} = \frac{2\text{ciclos}}{200\text{MHz}} = \frac{1}{100 \cdot 10^6} = 10\text{ns}$$

$$t_{\text{ciclo}} = \frac{1}{200\text{MHz}} = 0,5 \cdot 10^{-8} = 5 \cdot 10^{-9}\text{seg}$$

$$t_{\text{instr.}} = 2\text{ciclos} \cdot 5 \cdot 10^{-9} = 10 \cdot 10^{-9} = 10\text{ns}$$

$$t_{\text{CPU}} = \text{nº instr.} \cdot t_{\text{instr.}} = \frac{10\text{instr.}}{1\text{interrupción}} \cdot 10.000\text{interr.} = 100.000 \text{ instrucciones}$$

$$t_{\text{CPU}} = 100.000\text{instr.} \cdot 10\text{ns} = 1\text{ms}$$

Robo de E/S por CDMA → Roba el bus según el tiempo que se le permite.

Prog. → 50instr. RTI=10instr.
trabociclo = 10ns → se roba byte a byte

$$t_{\text{CPU}} = t_{\text{prog.}} + t_{\text{trabociclo}} + t_{\text{finalización}} = 500\text{ns} + 0,1\text{s} + 100\text{ns} \approx 0,1\text{seg}$$

$$t_{\text{prog.}} = 50\text{instr.} \cdot \frac{10\text{ns}}{1\text{instr.}} = 500\text{ns}$$

$$t_{\text{trabociclo}} = \text{nº robos} \cdot t_{\text{robo}} = 10^7 \cdot 10\text{ns} = 0,1\text{s}$$

nº robos = 10^7 robos

$$t_{\text{finalización}} = 10\text{instr.} \cdot \frac{10\text{ns}}{1\text{instr.}} = 100\text{ns}$$

Por ráfagas de bloques E/S por CDMA

ráfagas de 64Bytes $t_{\text{ráfaga}} = 10\text{ns}$

$$t_{\text{CPU}} = t_{\text{prog.}} + t_{\text{trabociclo}} + t_{\text{finalización}} \approx t_{\text{ráfagas}}$$

$$t_{\text{ráfagas}} = \text{nº ráfagas} \cdot t_{\text{ráfaga}} = 15625 \cdot 10\text{ns} = 15625\text{ms}$$

$$\text{nº ráfagas} = 10\text{MB}/64\text{B} = 1,5625 \cdot 10^5$$